Technical Documentation - ITSAR Mini ERP

1. Architecture Summary

The system uses a microservices architecture with independent deployment units and separate databases per service.

- API Gateway (`gateway`): single entry point
- Auth (`auth-service`): JWT login and identity
- Inventory (`inventory-service`): stock records
- Sales (`sales-service`): order management
- Purchasing (`purchasing-service`): supplier and PO flow
- CRM (`crm-service`): customer records
- Frontend (`frontend`): UI for primary flows

2. Service Boundaries and Datastores

- `auth-service` -> `/data/auth.db`
- `inventory-service` -> `/data/inventory.db`
- `sales-service` -> `/data/sales.db`
- `purchasing-service` -> `/data/purchasing.db`
- `crm-service` -> `/data/crm.db`

No database is shared between services.

3. Inter-Service Communication

REST over HTTP is used for synchronous process dependencies:

- Sales -> CRM internal endpoint for customer validation
- Sales -> Inventory internal endpoint for stock reservation
- Purchasing -> Inventory internal endpoint for restock on PO receiving

Internal calls are protected via `X-Service-Token`.

4. Authentication and Authorization

- JWT tokens generated by `auth-service`
- Protected endpoints in all functional services require bearer token
- Role checks:
  - Sales order creation: admin/manager/staff
  - Purchase order creation and receiving: admin/manager only

5. API Versioning

All APIs are versioned under `/api/v1/`.

6. Containerization and Orchestration

- Each service has a dedicated Dockerfile
- Full stack orchestration with Docker Compose (`docker-compose.yml`)
- Persistent SQLite files stored in mounted host volumes under `./data/*`

7. Technology Choices and Justification

- Python + FastAPI: fast development, automatic schema validation, OpenAPI support
- SQLite per service: simple isolated persistence for academic prototype
- Nginx frontend container: lightweight static delivery
- Docker Compose: reproducible local and VPS deployment path

8. Logging and Monitoring (Current + Next)

Current:

- Health endpoints per service (`/health`)
- HTTP status-driven error handling

Recommended extension:

- Structured JSON logs (request id, service, route, duration)
- Centralized log shipping (Loki/ELK)
- Metrics collection (Prometheus + Grafana)

9. Architecture Diagrams

- System context, container view, and sequence flows: `docs/architecture-diagrams.md`

10. API Specifications

OpenAPI files:

- `docs/openapi/auth-service.yaml`
- `docs/openapi/inventory-service.yaml`
- `docs/openapi/sales-service.yaml`
- `docs/openapi/purchasing-service.yaml`
- `docs/openapi/crm-service.yaml`

11. Data Models

- Complete data model reference: `docs/data-models.md`

12. Deployment

- Local deployment steps: `docs/deployment.md`
- Render production deployment steps: `docs/render-deployment.md`